A.E.D. Individual 3

Guillermo Román
guillermo.roman@upm.es

Julio García
juliomanuel.garcia@upm.es

Lars-Åke Fredlund
lfredlund@fi.upm.es

Tonghong Li
tonghong@fi.upm.es

Manuel Carro
mcarro@fi.upm.es

Francisco de Borja Lozano
francisco.lozano@upm.es

Normas

- Fechas de entrega y penalización:

Hasta el Martes 20 de octubre, 16:00 horas	0
Hasta el Miércoles 21 de octubre, 16:00 horas	20 %
Hasta el Jueves 22 octubre, 16:00 horas	40 %

  Después la puntuación máxima será 0
- Se comprobará plagio y se actuará sobre los detectados.
- Usad las horas de tutoría para preguntar sobre programación – son oportunidades excelentes para aprender.

Entrega

- Todos los ejercicios de laboratorio se deben entregar a través de

  `http://deliverit.fi.upm.es`

- El/los fichero(s) que hay que subir es/son `OrderedIterator.java`,`Utils.java`.
- La clase debe estar en el paquete aed.individual3
- La documentación de la API de `aedlib.jar` está disponible en

  `http://costa.ls.fi.upm.es/teaching/aed/docs/aedlib/`

Tarea 1: usar un Iterador

- Implementa el método

```
public static boolean
  isArithmeticSequence(Iterable<Integer> list)
```

  dentro el fichero `Utils.java`
- El método debe devolver `true` si `list` representa una secuencia aritmetica, y `false` si no.
- En una secuencia aritmetica la diferencia entre dos enteros consecutivos siempre es la misma.
- Por ejemplo, en `list=[1,3,5,7]` la diferencia entre dos enteros consecutivos siempre es 2 ($3-1$, $5-3$, y $7-5$); es una secuencia aritmetica.
- Notad que `list` puede contener elementos `null`, que no cuentan para el analisis si la lista es una secuencia aritmetica.

Ejemplos

```
list              isArithmeticSequence(list)
-----------------------------------------
[]                true
[null]            true
[i]               true (para cualquier i)
[i,j]             true (para cualquier i,j)
[1,2,3]           true
[1,null,2,3]      true
[1,2,null,4]      false
[1,2,2]           false
[1,3,5,7]         true
[-1,0,1]          true
```

Notas Específicos para `isArithMeticSequence`

- **No esta permitido usar bucles for-each**
- Notad que `list` puede contener elementos `null`

Tarea 2: Implementar un Iterador

- Termina la implementación de la clase `OrderedIterator` dentro el fichero `OrderedIterator.java`.
- Como la clase implementa la interfaz `Iterator<Integer>` debeis implementar obligatoriamente los métodos de un iterador: `hasNext`, `next`. No hace falta implementar `remove`.
- El constructor de dicha clase recibe un `PositionList<Integer> list` como argumento; los métodos del iterador debe devolver solamente los elementos dentro `list` que forman una secuencia *ordenada*.
- Concretamente, el primer entero dentro `list` siempre sera el primer elemento devuelto por `next()`. El segundo elemento devuelto por `next()` sera el siguente entero dentro `list` que es *mayor o igual que el primer entero*, etc.

Ejemplos

list	elementos devueltos por el iterador
[]	
[1]	1
[1,1]	1,1
[2,1]	2
[1,2,3]	1,2,3
[5,1,2,3,4,6,7,2]	5,6,7
[-1,0,-2]	-1,0

Notas Específicos para `OrderedIterator`

- **Se debe implementar un iterador eficiente que no crea otras estructuras de datos**.
- Es permitido, y necesario, de añadir nuevos atributos dentro la clase `OrderedIterator`.
- Notad que `list` no contiene elementos `null`.

Notas Generales

- El proyecto debe compilar sin errores y debe cumplirse la especificación de los métodos a completar
- Debe pasar todos los test `TesterInd3` correctamente sin mensajes de error
- **Nota:** una ejecución sin mensajes de error y que pase todas las pruebas **no** significa que la implementación sea correcta (es decir, que funcione bien para cada posible entrada)
- Todos los ejercicios se corrigen manualmente antes de dar la nota final